```
// Multi-year ROI projection calculations

export interface SingleYearResults {
  annualBenefits: number;
  annualCosts: number;
  timeSavings: number;
  qualitySavings: number;
  revenueSavings: number;
  hoursSavedWeekly: number;
  hoursSavedAnnually: number;
  ftesSaved: number;
  errorReductionPercent: number;
  errorsAvoidedAnnually: number;
  hoursReallocatedAnnually: number;
  [key: string]: number;
}

export interface CostBreakdown {
  implementationCost: number;
  annualSoftwareCost: number;
  trainingCosts: number;
  changeMgmtCosts: number;
}

export interface YearlyProjection {
  year: number;
  benefits: number;
  costs: number;
  netBenefit: number;
  cumulativeBenefits: number;
  cumulativeCosts: number;
  cumulativeNetBenefit: number;
  roi: number;
  cumulativeROI: number;
  timeSavings: number;
  qualitySavings: number;
  revenueSavings: number;
}

export interface MultiYearResults {
  yearlyProjections: YearlyProjection[];
  totalBenefits5Year: number;
  totalCosts5Year: number;
  netBenefit5Year: number;
  roi5Year: number;
  paybackMonths: number;
```

```
  year1RampFactor: number;
}

/**
 * Calculate benefit multiplier for a given year based on ramp-up period
 * Handles ramp-up periods >12 months correctly
 *
 * Business Rule: During ramp-up months, assume 50% average efficiency.
 * After ramp-up completes, assume 100% efficiency.
 *
 * @param year - The year number (1, 2, 3, etc.)
 * @param rampUpMonths - Total ramp-up period in months
 * @param efficiencyAdjustedRampFactor - Optional pre-calculated Year 1 ramp factor with
efficiency applied
 * @returns Benefit multiplier for that year (0.0 to 1.0)
 */
function calculateYearBenefitMultiplier(
  year: number,
  rampUpMonths: number,
  efficiencyAdjustedRampFactor?: number
): number {
  if (rampUpMonths <= 12) {
    // Ramp-up completes within Year 1
    // Formula: (full production months + 50% during ramp-up months) / 12
    if (year === 1) {
      return efficiencyAdjustedRampFactor !== undefined
        ? efficiencyAdjustedRampFactor
        : (12 - rampUpMonths + (rampUpMonths * 0.5)) / 12;
    }
    // Years 2+ get full benefits
    return 1.0;
  } else {
    // Ramp-up extends beyond Year 1
    const yearStartMonth = (year - 1) * 12 + 1; // First month of this year (1-indexed)
    const yearEndMonth = year * 12; // Last month of this year

    // Determine how many months of this year are still ramping up
    const lastRampMonth = rampUpMonths; // Ramp-up completes at end of this month

    if (yearStartMonth > lastRampMonth) {
      // This year starts after ramp-up completed - full 100% benefits
      return 1.0;
    }

    // Calculate how many months of this year are still ramping up
    const monthsStillRamping = Math.min(12, Math.max(0, lastRampMonth - yearStartMonth + 1));
```

```
    const monthsAtFullCapacity = 12 - monthsStillRamping;

    if (monthsAtFullCapacity === 0) {
      // Entire year is still within ramp-up period - 50% average efficiency
      if (year === 1 && efficiencyAdjustedRampFactor !== undefined) {
        // Year 1 uses efficiency-adjusted factor if provided
        return efficiencyAdjustedRampFactor;
      }
      return 0.5;
    }

    // Ramp-up completes partway through this year
    // Weighted average: ramp-up months at 50%, full production months at 100%
    const yearAvg = (monthsStillRamping * 0.5 + monthsAtFullCapacity * 1.0) / 12;

    return yearAvg;
  }
}

/**
 * Calculate multi-year ROI projections with ramp-up modeling
 *
 * @param baseResults - Single year calculation results (full run-rate)
 * @param costBreakdown - Detailed cost breakdown for multi-year modeling
 * @param rampUpMonths - Implementation ramp-up period (1-36 months supported)
 * @param years - Number of years to project (default: 5)
 * @param efficiencyAdjustedRampFactor - Optional pre-calculated ramp factor with efficiency
applied (for Year 1 only)
 * @returns Multi-year projection results
 */
export function calculateMultiYearROI(
  baseResults: SingleYearResults,
  costBreakdown: CostBreakdown,
  rampUpMonths: number = 6,
  years: number = 5,
  efficiencyAdjustedRampFactor?: number
): MultiYearResults {
  const yearlyProjections: YearlyProjection[] = [];

  // Year 1 ramp-up factor using the correct calculation for all ramp-up periods
  // This ensures consistency between reported factor and actual benefits calculation
  const year1RampFactor = calculateYearBenefitMultiplier(1, rampUpMonths,
efficiencyAdjustedRampFactor);

  console.log('🔧 Multi-Year Calculation:', {
    rampUpMonths,
```

```
    year1RampFactor: `${(year1RampFactor * 100).toFixed(1)}%`,
    fullAnnualBenefits: baseResults.annualBenefits,
    year1Benefits: baseResults.annualBenefits * year1RampFactor,
    year1Costs: costBreakdown.implementationCost + costBreakdown.annualSoftwareCost +
costBreakdown.trainingCosts + costBreakdown.changeMgmtCosts
  });

  let cumulativeBenefits = 0;
  let cumulativeCosts = 0;
  let paybackMonths = 0;
  let...
```